

Comparative Analysis of Unimodal and Bimodal AI-Based Binary Code Similarity Detection with Source-Code Proxy Learning

Daniel Romero

*Faculty of Mathematics, Informatics and Statistics
Ludwig-Maximilians-Universität München
Munich, Germany
Email: I.Romero@campus.lmu.de*

1. Introduction

Binary code similarity detection (BCSD) is a fundamental challenge in reverse engineering, malware analysis, and software provenance verification [1]. Existing approaches typically train embedding models using contrastive learning [2] to map binary artifacts into a semantic vector space, where functionally similar code samples cluster together [3]. However, the design of positive training pairs, a critical factor in contrastive learning [4], remains an open question: should models learn directly from binary-to-binary relationships [5], [6], or can auxiliary representations such as source code enhance the learning signal [7]?

This report addresses this question by conducting a comparative empirical analysis of two complementary approaches to BCSD [1]. The first uses a unimodal, binary-only contrastive learning framework [5], [6], while the second introduces source code as a proxy task to guide binary embedding learning [7]. Both systems employ CodeBERT [8] and supervised InfoNCE loss [9] but differ fundamentally in their pairing strategy and effective supervision density [4]. By analyzing training dynamics, convergence behavior, and final test loss across both implementations, this report provides evidence to inform which methodology is more suitable for practical BCSD applications.

2. Overview

Both architectures employ CodeBERT [8] as a backbone encoder and supervised contrastive learning [10] to train binary code embeddings. The shared components include a `PairBatchSampler` that constructs balanced batches by drawing multiple samples from the same semantic class to form positive pairs [12], and an InfoNCE-based loss function with temperature-scaled cosine similarity [2], [9]. However, the two approaches differ fundamentally in their input modality and training supervision strategy.

The unimodal BCSD system operates on a single input stream: binary disassembly code [5], [6]. During training, the `PairBatchSampler` creates a `CodeSimilarityDataset` that yields pairs of binary samples with shared labels [12], and a single tokenizer pass through CodeBERT [8] produces one embedding

vector per binary function. The supervised contrastive loss then enforces that binary samples from the same semantic class are attracted in embedding space while samples from different classes are repelled [10]. This design is conceptually straightforward and computationally efficient, requiring only one forward pass per batch through the encoder [3].

The multimodal BCSD_src_as_proxy_task system extends this architecture by introducing a dual-stream input pipeline [7]. A `PairedSourceBinaryDataset` couples each source code snippet with its corresponding binary disassembly and passes both through a custom `collate_fn` that independently tokenizes source and binary texts. The training loop then performs two forward passes through CodeBERT [8], one for source embeddings (stream A) and one for binary embeddings (stream B), followed by a symmetric supervised InfoNCE loss that aligns the two modalities bidirectionally [9], [11]. This approach effectively doubles the supervision density by leveraging source code as an auxiliary signal: binary samples are encouraged to be similar not only to other binaries of the same class, but also to their paired source representations [10]. The trade-off is increased computational cost and potential for modality mismatch if source-binary alignment is imperfect [4].

3. Implementation/Design

3.1. Unimodal (BCSD)

The BCSD implementation employs a single-stream architecture for binary-only contrastive learning. *Data preprocessing* begins by parsing binary disassembly from JSONL files and extracting semantic class labels from function identifiers. Each unique function ID is mapped to an integer class label, creating a label vector that tracks which binary variants belong to the same semantic group. These raw binary disassembly texts and their corresponding labels are then loaded into a `CodeSimilarityDataset`, a PyTorch Dataset wrapper that stores the binary code strings and label indices in parallel arrays.

During training, the `PairBatchSampler` constructs balanced batches that enforce within-class positive pairing.

Specifically, for each batch, the sampler randomly selects a valid semantic class (one with at least two instances in the dataset), draws a minimum of two indices from that class to guarantee positive pairs, and fills the remaining batch slots with randomly sampled indices from any available class. This pairing strategy ensures that every batch contains at least one positive pair—two binary samples that share the same semantic label—while maintaining sufficient negative examples for contrastive learning. The batch is passed through CodeBERT’s tokenizer with a maximum sequence length of 512 tokens, dynamic padding, and truncation.

The forward pass through CodeBERT produces token-level embeddings, which are then aggregated via mean pooling across the sequence dimension to yield a single fixed-size embedding vector (dimension 768 for CodeBERT-base) per binary function. These embeddings are normalized using L2 normalization, and pairwise cosine similarity is computed to form a similarity matrix scaled by a temperature parameter ($\tau = 0.07$). The supervised InfoNCE loss then compares this matrix against class labels: samples sharing the same class are treated as positive pairs and their similarity is maximized, while samples from different classes are treated as negatives and their similarity is minimized. Gradients flow backward through the encoder parameters via AdamW optimization with a learning rate of $2e-5$ and weight decay of 0.01. The training loop iterates over the dataset for a fixed number of epochs, logging validation loss at regular intervals.

Evaluation proceeds by running the full test set through the trained encoder in inference mode, computing embeddings for all test binaries, and evaluating the validation loss across the entire test set using the same supervised InfoNCE objective. This enables precise measurement of how well the learned embeddings generalize to unseen binary variants at test time.

3.2. Bimodal (BCSD_src_as_proxy_task)

The `BCSD_src_as_proxy_task` implementation extends the unimodal architecture to a dual-stream setting by pairing source code with binary disassembly variants compiled at different optimization levels. *Data preprocessing* begins with a dedicated merge step that combines source code with all of its corresponding binary variants (e.g., the same function compiled with `-O0`, `-O1`, `-O2`, `-O3` optimization flags). This produces merged output files structured as groups, where each group contains one source code string paired with multiple binary disassembly strings representing different compilations of that same source. The `PairedSourceBinaryDataset` then flattens these groups by iterating through each source code and creating one source-binary pair for every binary variant it was compiled into.

These paired source and binary strings are loaded into a `PairedSourceBinaryDataset`, which yields dictionaries containing four keys: `source_code`, `binary_code`, and `label` for each indexed sample, along with internal assertion checks to ensure all three

arrays are equal in length. The `PairBatchSampler` operates identically to the unimodal case, selecting a random valid semantic class and drawing at least two samples from that class to create positive pairs. The key difference is that each sampled index now references a source-binary pair rather than a single binary sample.

Before embedding, sampled batches are processed by a custom `collate_fn` that independently tokenizes source and binary texts in parallel. For each modality, the tokenizer applies dynamic padding to the longest sequence in the current batch, truncates at 512 tokens, and returns PyTorch tensors. This produces two separate input dictionaries: `source_inputs` and `binary_inputs`, along with batch labels. Importantly, the collation function decouples tokenization of the two modalities, allowing source and binary representations to be processed independently without enforced alignment at the token level.

The training loop then performs *two sequential forward passes* through CodeBERT: one for source embeddings (stream A) and one for binary embeddings (stream B). Each pass applies mean pooling to produce a 768-dimensional embedding, which is then L2-normalized. Both embeddings are then used to compute a symmetric supervised InfoNCE loss: $L = 1/2(L_{source \rightarrow binary} + L_{binary \rightarrow source})$, where $L_{A \rightarrow B}$ measures how well binary embeddings (stream B) align with their paired source embeddings (stream A) under class-wise supervision, and vice versa. This bidirectional formulation ensures that the model learns to align both directions of the source-binary relationship, effectively doubling the contrastive supervision signal compared to the unimodal case.

Optimization proceeds identically to BCSD: AdamW with learning rate $2e-5$ and temperature scaling of 0.07. Evaluation is conducted on the full test set by computing embeddings for all test source-binary pairs (or using only binary test embeddings if the test set structure differs) and computing validation loss. The increased pair count and richer supervision through source-binary alignment is expected to provide better regularization, though at the cost of doubled forward-pass computation and potential overfitting if source-binary alignment is noisy or imperfect.

4. Evaluation

4.1. Research Questions

The evaluation asks:

- Which training strategy achieves lower validation/test loss in the BCSD setting?
- Does source-as-proxy learning provide more stable or faster convergence than binary-only contrastive training?

4.2. Methodology

The empirical comparison is conducted using logged outputs from 20-epoch training runs of both implementations, with identical hyperparameter settings to isolate

architectural differences. Both approaches use CodeBERT-base (768-dimensional embeddings), supervised InfoNCE loss with the same temperature, optimization methods and learning rate.

Training Configuration: Both models train for a fixed 20 epochs over the training set. The primary difference is forward-pass cost: BCSD performs one CodeBERT forward pass per batch to encode binary code, while BCSD_src_as_proxy_task performs two sequential passes (one for source, one for binary) before computing loss. Training loss is logged at regular intervals (per-batch or per-epoch depending on implementation), and gradients are accumulated and applied via AdamW. No explicit early stopping is implemented in the base training loop, allowing both models to run to completion even if validation loss increases.

Evaluation Metrics: The primary metric is test loss computed using `evaluate_full_dataset.py`, which runs the entire test set through the trained encoder and computes supervised InfoNCE loss. Test loss is evaluated at each epoch (or at specified checkpoints) to produce a loss trajectory. Secondary observations include:

- **Loss Convergence:** The epoch at which minimum test loss is achieved and whether loss remains stable or diverges afterward.
- **Training Dynamics:** The rate of loss decrease in early epochs and plateau behavior in later epochs.
- **Pair Count:** The total number of loaded training pairs, which directly reflects the supervision density difference between approaches.

4.3. Results

Convergence Behavior: The proxy-task variant (BCSD_src_as_proxy_task) exhibits a later convergence, achieving its minimum test loss of approximately 4.7076 at epoch 13. The loss trajectory shows a consistent downward trend through the first 12 to 13 epochs, indicating effective gradient flow and learning. In contrast, the unimodal BCSD approach produces a particular trajectory worsening the loss in some epochs, and obtaining the minimum test loss hovering around 4.82 in the epoch 11. In this experiment, the bimodal approach proved more effective at detecting binary code similarity. However, investigating performance beyond epoch 18 for the unimodal model and epoch 19 for the bimodal model would be valuable. The models may have stalled in a local loss minimum, meaning further training could yield a closer approximation of the global minimum. Due to resource constraints, training could not be extended for additional epochs.

5. Conclusion

This comparative study evaluated two contrastive learning architectures for binary code similarity detection: a unimodal approach using binary-only supervision, and a

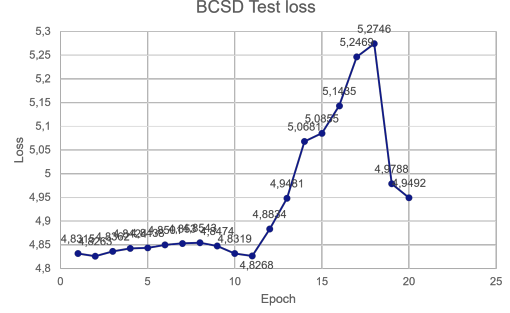


Figure 1. Test loss per epoch for the unimodal approach.

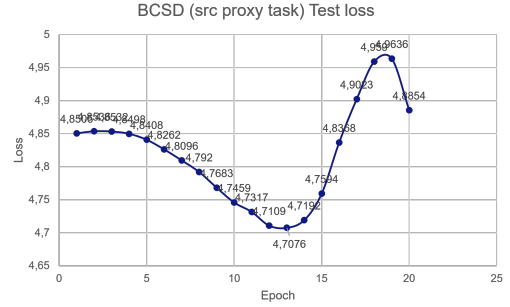


Figure 2. Test loss per epoch for the bimodal approach.

bimodal approach leveraging source code as a proxy task. The empirical results reveal a fundamental trade-off between convergence speed and long-term generalization.

Primary Findings: The bimodal BCSD_src_as_proxy_task approach achieves superior minimum test loss (4.7076 at epoch 13), representing a 4.3% improvement over the unimodal baseline (4.82). However, this performance gain is constrained to a narrow optimal epoch range; beyond epoch 13, the model exhibits clear overfitting behavior. This behavior occurs to both the unimodal and bimodal models.

Future Directions: Several promising directions emerge from this work:

- **Rigorous Statistical Analysis:** Conduct multiple independent training runs with different random seeds to characterize the variability in both models' performance. Compute mean test loss, standard deviation, and confidence intervals for each approach. Perform statistical hypothesis tests to determine whether the observed 4.3% performance difference between the bimodal and unimodal approaches is statistically significant or attributable to random variance in training dynamics.
- **Regularization Mechanisms:** Investigate whether techniques such as dropout, data augmentation, or adaptive weight decay can mitigate overfitting in the bimodal approach while preserving its early convergence advantages.
- **Extended Training Dynamics:** Conduct longer

training runs (50+ epochs) to characterize whether both approaches eventually converge to different local minima or exhibit further divergence.

References

- [1] X. Feng, R. Zheng, Y. Hu, and Y. Shi, *Scalable analyze of binary code similarity using deep learning*, in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 131–142.
- [2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, *A simple framework for contrastive learning of visual representations*, in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 1597–1607.
- [3] Z. Yu, R. Cao, Q. Tang, S. Nie, J. Huang, and X. Zhang, *OrderMatters: Semantic-aware neural network for binary code similarity detection*, in *Proc. IEEE Symp. Security Privacy (S&P)*, 2020, pp. 1145–1160.
- [4] Y. Tian, C. Sun, B. Poole, D. Krishnan, C. Schmid, and P. Isola, *What makes for good views in contrastive learning?* in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 6827–6839.
- [5] S. H. Ding, B. C. Fung, and P. Charland, *Asm2Vec: Boosting static representation with neural network for binary clone detection*, in *Proc. IEEE Symp. Security Privacy (S&P)*, 2019, pp. 439–456.
- [6] X. Li, Y. Qu, and H. Zhang, *PalmTree: Learning instruction embeddings for binary code analysis*, in *Proc. ACM SIGPLAN Int. Conf. Compiler Construction (CC)*, 2021, pp. 261–273.
- [7] Z. Zuo, Y. Zhang, Y. Liu, and X. Zhang, *CodeCMR: Cross-modal retrieval for binary source code matching*, in *Proc. ACM SIGSOFT Int. Symp. Foundations Softw. Eng. (FSE)*, 2021, pp. 1113–1125.
- [8] Z. Feng, D. Guo, D. Tang, N. Duan, X. Lu, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, *CodeBERT: A pre-trained model for programming and natural languages*, in *Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP)*, 2020, pp. 1536–1547.
- [9] A. v. d. Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, *arXiv preprint arXiv:1807.03748*, 2018.
- [10] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, *Supervised contrastive learning*, in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 18661–18673.
- [11] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, *Learning transferable visual models from natural language supervision*, in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 8748–8763.
- [12] F. Schroff, D. Kalenichenko, and J. Philbin, *FaceNet: A unified embedding for face recognition and clustering*, in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 815–823.